Package 'rProject'

November 6, 2026

Title Setup R Project

Version 0.0.63

Description A collection of functions to setup R projects.

URL https://github.com/ijapesigan/rProject/, https://ijapesigan.github.io/rProject/

BugReports https://github.com/ijapesigan/rProject/issues

License MIT + file LICENSE

Encoding UTF-8

LazyData true

Roxygen list(markdown = TRUE)

VignetteBuilder knitr

Depends R (>= 3.5.0)

Imports BiocManager, cffr, covr, devtools, distro, jsonlite, knitr, lintr, pak, pkgdown, quarto, Rcpp, remotes, rmarkdown, roxygen2, rprojroot, stats, styler, tinytex, utils

Suggests testthat

Config/roxygen2/version 8.1.1.9000

NeedsCompilation no

Author Ivan Jacob Agaloos Pesigan [aut, cre, cph] (ORCID: <https://orcid.org/0000-0003-4818-8420>)

Maintainer Ivan Jacob Agaloos Pesigan <ijapesigan@gmail.com>

Contents

AI 3
AIGlobal 4
Bib 5
BibLib 6
Binary 7

Build . . . 7
BuildIgnore . . . 8
CFF . . . 9
Clean . . . 10
CleanPkg . . . 10
CleanProj . . . 11
CleanTinyTex . . . 12
ConfigFiles . . . 13
Coverage . . . 13
DataAnalysis . . . 14
DataCompress . . . 15
DataManifest . . . 15
DataPath . . . 16
DataProcess . . . 17
Dependencies . . . 18
Detritus . . . 18
Environment . . . 19
EnvironmentProject . . . 20
EnvironmentTokens . . . 20
InstallLocal . . . 21
LatexCompile . . . 22
LatexKnit . . . 23
LatexMake . . . 23
LatexTikzCompile . . . 24
LibAdd . . . 25
LibCheck . . . 26
LibFormat . . . 27
LibPaths . . . 28
License . . . 29
Lint . . . 29
Lock . . . 30
Log . . . 31
Manual . . . 32
PkgDevel . . . 33
PkgDevelGitHub . . . 34
PkgProject . . . 34
PkgProjectBioconductor . . . 35
PkgProjectGitHub . . . 36
PkgProjectVersion . . . 36
PkgRepo . . . 37
Profile . . . 38
Project . . . 39
Public . . . 39
Quarto . . . 40
ReadMe . . . 41
SecretScan . . . 41
Site . . . 42
SizeCheck . . . 43

Spec 44
Style 45
Tags 45
Template 46
TinyTex 48
Tree 48
Verify 49
VignettesPrecompile 51
z 52

Index **53**

`AI` *Write the Project's AI Instructions*

Description

Writes the instruction files that AI tools read in this project, on top of the global instructions installed by `AIGlobal()`.

Usage

```
AI(path, type = NULL)
```

Arguments

`path`	Character string. Project path.
`type`	Character vector. Project types: any of `"package"`, `"cran"` (a package on CRAN or headed there; with `"package"`), `"manuscript"`, `"simulation"`, `"library"`, `"talk"`, and `"container"`.

Details

- `.setup/ai/project.md`: project facts that the user maintains. Created once and never overwritten.
- `.setup/todo/`: the user's to-do list, `todo.txt`, which agents read when planning and never edit, with a `.gitignore` that keeps the folder out of the public repository. Created only if the folder does not exist.
- `AGENTS.md` (Codex, Copilot CLI, Claude through `CLAUDE.md`), `CLAUDE.md` (Claude Code), and `.github/copilot-instructions.md` (Copilot in the IDE): generated, and rewritten when they change.

All of them are private (gitignored and excluded from the package build).

The project type comes from `type`, else from an `ai` list in `.setup/scripts/project.R` (e.g., `ai <- list(type = c("manuscript", "simulation"))`), else from the template type in `.setup/scripts/project.R` (`type <- "cran"`; see `Template()`), else it is detected: `"simulation"` if `.sim/README.md` (which describes the simulation) exists, `"manuscript"` if a `*-manuscript*.Rtex` exists or the name starts

with man, "package" and "cran" for a cran-* folder (its project.R names the package), "library" for lib-*, "talk" for talks*, "container" for docker-*, otherwise "package".

Where the project lives (the private GitHub repository, the Overleaf project, and the public GitHub repository) comes from .setup/repo/, which Project() creates when it is missing.

The user chooses the Apptainer image and runs apptainer exec; the instructions tell agents not to pick an image.

Value

Returns the paths of the files that were written, invisibly.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
AI(path = getwd())

## End(Not run)
```

AIGlobal *Install Global AI Instructions*

Description

Copies the global AI instructions and Claude settings from .setup/ai/global in the project to the configuration folders of Claude Code, Codex, and GitHub Copilot in the home directory. Files are written only when their content changed. Existing files that differ are backed up with the suffix .bak.

Usage

```
AIGlobal(path, home = Sys.getenv(x = "HOME"))
```

Arguments

path	Character string. Project path.
home	Character string. Home directory.

Details

Source	Installed to
AGENTS.md	~/.claude/CLAUDE.md
AGENTS.md	~/.codex/AGENTS.md
AGENTS.md	~/.copilot/copilot-instructions.md
claude-settings.json	~/.claude/settings.json

claude-settings.json is merged into an existing settings.json: its keys win, permission rules are added to the existing ones, and other keys are kept.

Value

Returns the paths of the files that were written.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
AIGlobal(path = getwd())

## End(Not run)
```

Bib *Consolidate .bib Files into a Single File*

Description

Joins the .bib files in .setup/latex/bib into bib.bib, verbatim and in file-name order, and copies it to .setup/quarto/bib/quarto.bib and vignettes/vignettes.bib when those folders exist. Without sources (a clone of the repository, which has only bib.bib), the existing bib.bib is copied. The source files are never rewritten: in lib-* projects they are the entries themselves (one entry per file, ASCII with LaTeX accents). LibCheck() validates them with biber.

Usage

```
Bib(path, bib_lib = FALSE)
```

Arguments

path	Character string. Project path.
bib_lib	Logical. If bib_lib = TRUE, build bib.bib from the public lib-* GitHub repositories (make bib does so in rProject, make bibrproject anywhere). If none can be cloned, the previous bib.bib is kept.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Bib(path = getwd())

## End(Not run)
```

BibLib *Copy .bib Files from lib-* GitHUb Repositories*

Description

Copy .bib Files from lib-* GitHUb Repositories

Usage

```
BibLib(path)
```

Arguments

path	Character string. Project path.

Value

Returns a logical value from copying the bibliography.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
BibLib(path = getwd())

## End(Not run)
```

Binary *Setup Binaries not Included in the Package*

Description

Runs (sources) every R script in `.setup/bin/`, at `make project`; for example, a script that builds a bespoke binary the project needs. `.setup/bin/` is optional: `make` does not create it, and without it `Binary()` does nothing.

Usage

```
Binary(path)
```

Arguments

`path`	Character string. Project path.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Binary(path = getwd())

## End(Not run)
```

Build *Build Package*

Description

Build Package

Usage

```
Build(path, vignettes = FALSE, dependencies = FALSE)
```

Arguments

path	Character string. Project path.
vignettes	Logical. Build vignettes.
dependencies	Logical. Install package dependencies.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Build(path = getwd())

## End(Not run)
```

BuildIgnore *Build* .Rbuildignore

Description

Build .Rbuildignore

Usage

```
BuildIgnore(path, project, add = NULL)
```

Arguments

path	Character string. Project path.
project	Character string. Project name.
add	Character vector. Items to add to .Rbuildignore.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
BuildIgnore(path = getwd(), project = "myProject")

## End(Not run)
```

CFF *Build* CITATION.cff

Description

Build CITATION.cff

Usage

```
CFF(path)
```

Arguments

path	Character string. Project path.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
CFF(path = getwd())

## End(Not run)
```

Clean *Clean Files Generated on Build*

Description

Delete all output files that are generated on automated build.

Usage

```
Clean(path, push = FALSE)
```

Arguments

path	Character string. Project path.
push	Logical. If push = TRUE clean in preparation for git push: generated files are kept, and files matching *secrets* are deleted, except in hidden folders such as .setup.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Clean(path = getwd())

## End(Not run)
```

CleanPkg *Clean R Packages*

Description

Remove the project's R library (.library). Packages outside the project (the image's site library, the user library) are never touched.

Usage

```
CleanPkg(path)
```

Arguments

path	Character string. Project path.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
CleanPkg(path = getwd())

## End(Not run)
```

CleanProj *Clean Project*

Description

Remove all files except `project.Rproj` and, optionally, `.library`.

Usage

```
CleanProj(path, lib = FALSE, sim = FALSE, sif = FALSE)
```

Arguments

path	Character string. Project path.
lib	Logical. If `lib = TRUE`, delete `.library`. If `lib = FALSE`, `.library` is not deleted.
sim	Logical. If `sim = TRUE`, delete `.sim`. If `sim = FALSE`, `.sim` is not deleted.
sif	Logical. If `sif = TRUE`, delete `.sif`. If `sif = FALSE`, `.sif` is not deleted.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
CleanProj(path = getwd())

## End(Not run)
```

CleanTinyTex *Clean* ~/.TinyTex

Description

Uninstall TinyTex.

Usage

```
CleanTinyTex(path)
```

Arguments

path	Character string. Project path.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
CleanTinyTex(path = getwd())

## End(Not run)
```

ConfigFiles *Setup Config Files*

Description

Clones a public `dotfiles` GitHub repository and runs `make` in it. The clone ignores the global Git configuration, so URL rewrites to SSH host aliases do not apply and the repository is fetched anonymously over HTTPS. This also works inside a container without access to SSH keys.

Usage

```
ConfigFiles(repo = "ijapesigan/dotfiles")
```

Arguments

`repo`	Character string. GitHub repository in the form `owner/name`.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
ConfigFiles()

## End(Not run)
```

Coverage *Code Coverage*

Description

Runs `covr::package_coverage()` on the package. To leave files out, put covr's `.covrignore` at the project root (and add it to `.Rbuildignore` with `ignore` in `.setup/scripts/project.R`).

Usage

```
Coverage(path)
```

Arguments

`path`	Character string. Project path.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Coverage(path = getwd())

## End(Not run)
```

`DataAnalysis` *Data Analysis*

Description

Execute R scripts in `data-analysis`.

Usage

```
DataAnalysis(path, clean = FALSE)
```

Arguments

`path`	Character string. Project path.
`clean`	Logical. Remove all `.Rds` files in `data-process` and `data-analysis` before running.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
DataAnalysis(path = getwd())

## End(Not run)
```

DataCompress *Compress Data Files*

Description

Compresses data files in place with xz (e.g., `data.txt` becomes `data.txt.xz`), checks that the compressed copy reads back byte for byte, and then removes the original. Compressed files are binary, so they do not count as editable text in Overleaf (see `SizeCheck()`). Scripts that find files with `DataPath()` keep working unchanged.

Usage

```
DataCompress(path, files)
```

Arguments

`path`	Character string. Project path.
`files`	Character vector. Files to compress, relative to `path`.

Value

Returns the paths of the compressed files, invisibly.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
DataCompress(path = getwd(), files = "data.csv")

## End(Not run)
```

DataManifest *Write the Data Manifest*

Description

Writes `.setup/ai/data.md` (private), with one row for each file in the data tiers (`.setup/data-raw`, `.setup/data-private`, `.setup/data-external`): the file, its tier, size, MD5, status, the source and location (filled in by the user and kept on rewrite), and the scripts that use it.

Usage

```
DataManifest(path)
```

Arguments

path	Character string. Project path.

Details

External files are local only. One that is listed but missing on this machine is kept, marked "not on this machine"; one whose MD5 differs from the recorded one is marked "changed".

Value

Returns the manifest as a data frame, invisibly.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
DataManifest(path = getwd())

## End(Not run)
```

DataPath *Find a Data File in the Data Tiers*

Description

Finds a data file by its plain name in the data tiers, in this order: `.setup/data-raw` (shareable), `.setup/data-private` (restricted; never public), and `.setup/data-external` (local only). The file can be stored as is or compressed (`.xz` or `.gz`); `utils::read.table()` and similar functions read compressed files directly.

Usage

```
DataPath(path, file, plain = FALSE)
```

Arguments

path	Character string. Project path.
file	Character string. Plain file name, e.g., `"data.txt"`, even if it is stored as `data.txt.xz`.
plain	Logical. If `TRUE`, return a path to an uncompressed copy.

Details

With `plain = TRUE`, a compressed file is unpacked into `.setup/data-cache` (local only), for tools that need plain text, such as Mplus. The copy is refreshed when the compressed file changes.

Value

Returns the path of the file.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
DataPath(path = getwd(), file = "data.csv")

## End(Not run)
```

`DataProcess` *Process Data*

Description

Execute R scripts in `data-process`.

Usage

```
DataProcess(path, clean = FALSE)
```

Arguments

`path`	Character string. Project path.
`clean`	Logical. Remove all `.Rds` files in `data-process` before running.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
DataProcess(path = getwd())

## End(Not run)
```

Dependencies *Copy Dependencies to from* .setup/r-dependencies *to* R

Description

Copy Dependencies to from .setup/r-dependencies to R

Usage

```
Dependencies(path)
```

Arguments

path	Character string. Project path.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Dependencies(path = getwd())

## End(Not run)
```

Detritus *Create* detritus/tex/tex-fig-*

Description

Create detritus/tex/tex-fig-*

Usage

```
Detritus(path)
```

Arguments

path	Character string. Project path.

Value

Returns a list with the detritus folder and the paths of the `.tex` files.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Detritus(path = getwd())

## End(Not run)
```

`Environment` *Setup* `.Renviron`

Description

Setup `.Renviron`

Usage

```
Environment(path, project, tokens)
```

Arguments

`path`	Character string. Project path.
`project`	Character string. Project name.
`tokens`	Named vector of tokens.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Environment(
  path = getwd(),
  project = "myProject",
  tokens = c(GITHUB_PAT = Sys.getenv(x = "GITHUB_PAT"))
)

## End(Not run)
```

EnvironmentProject *Set* PROJECT *Environment Variable*

Description

Set PROJECT Environment Variable

Usage

```
EnvironmentProject(project)
```

Arguments

project	Character string. Project name.

Value

Returns project name.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
EnvironmentProject(project = "myProject")

## End(Not run)
```

EnvironmentTokens *Set Tokens*

Description

Set Tokens

Usage

```
EnvironmentTokens(tokens)
```

Arguments

tokens	Named vector of tokens.

Value

Returns vector of secret tokens.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
EnvironmentTokens(tokens = c(GITHUB_PAT = Sys.getenv(x = "GITHUB_PAT")))

## End(Not run)
```

InstallLocal *Install Applications on Local Machine*

Description

Install Applications on Local Machine

Usage

```
InstallLocal(
  what = c("tmux", "julia", "mpdemo", "lazygit", "fastfetch", "quarto"),
  all = FALSE,
  force = FALSE
)
```

Arguments

what	Character string. Vector of applications to install. Valid values include "tmux", "julia", "mpdemo", "lazygit", "fastfetch", and "quarto".
all	Logical. If all = TRUE, ignore what and install all valid what values.
force	Logical. If force = TRUE, install whether or not the application is available in the system.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
InstallLocal()

## End(Not run)
```

`LatexCompile` *Compile Output of* `LatexKnit`

Description

Runs `latexmk` on every `.tex` file, with 10 million extra words of pdfTeX main memory (`extra_mem_bot`), so that a long bibliography compiles. A document whose last run failed (fatal error, TeX capacity exceeded) stops `make latex` in GitHub Actions and gives a warning locally: `latexmk -f` would otherwise leave a stub PDF and succeed.

Usage

```
LatexCompile(path, clean = FALSE)
```

Arguments

`path`	Character string. Project path.
`clean`	Logical. Clean `latexmk` output except `*.pdf`

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
LatexCompile(path = getwd())

## End(Not run)
```

LatexKnit *Knit* .setup/latex/*.Rtex

Description

Knit .setup/latex/*.Rtex

Usage

```
LatexKnit(path)
```

Arguments

path	Character string. Project path.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
LatexKnit(path = getwd())

## End(Not run)
```

LatexMake *Compile Output of* LatexKnit

Description

Compile Output of LatexKnit

Usage

```
LatexMake(path, clean = FALSE)
```

Arguments

path	Character string. Project path.
clean	Logical. Clean latexmk output except *.pdf

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
LatexMake(path = getwd())

## End(Not run)
```

`LatexTikzCompile` *Compile Tikz*

Description

Compile Tikz

Usage

```
LatexTikzCompile(path, clean = FALSE)
```

Arguments

`path`	Character string. Project path.
`clean`	Logical. Clean `latexmk` output except `*.pdf`

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
LatexTikzCompile(path = getwd())

## End(Not run)
```

`LibAdd` *Add an Entry to the lib-* Library from a DOI*

Description

Drafts a biblatex entry from the DOI's Crossref record, in the house layout, and names the repository and file it belongs in:

Usage

```
LibAdd(doi, root, key = NULL, title = NULL, fields = NULL, write = FALSE)
```

Arguments

`doi`	Character string. The DOI.
`root`	Character string. The folder with local clones of the `lib-*` projects.
`key`	Character string or `NULL`. A key to use instead of the drafted one (e.g., with a suffix).
`title`	Character string or `NULL`. A title to use instead of the drafted one.
`fields`	Named character vector or `NULL`. Fields to set or replace (e.g., full author names or an abstract from the publisher's page).
`write`	Logical. If `TRUE`, write the file and run `LibCheck()` on its repository.

Details

- the entry type and the repository type from the Crossref type;
- `date` from the print issue (`published-print`), else the journal issue, else the first publication; the key year and the decade repository follow it;
- names and text with LaTeX accents, and `\%` and `\&` escaped;
- the key as `Family-Family-Family[-etal]-Year`, accents dropped;
- the abstract without its JATS tags;
- the ISSN and publisher reused from the library's other entries for the same journal;
- the title in a sentence-case draft.

It never invents a field: what Crossref does not have is left out. It lists what it cannot decide (initials only, an online-first date, a missing abstract, the title's proper nouns, a key already in use) as questions for the user.

Value

Returns a list with `file`, `text`, and `questions`, invisibly.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
LibAdd(doi = "10.1000/xyz123", root = "...")

## End(Not run)
```

`LibCheck` *Check the Entries of a lib-* Library*

Description

Checks every entry file in `.setup/latex/bib` of a `lib-<decade>-<type>` project against the house conventions.

Usage

```
LibCheck(path, root = NULL, error = FALSE)
```

Arguments

`path`	Character string. Project path.
`root`	Character string or `NULL`. A folder with local clones of the other `lib-*` projects, to find keys and DOIs used in more than one repository. If `NULL`, only this project is checked.
`error`	Logical. If `TRUE`, stop when there is an ERROR.

Details

- **ERROR:** the file does not parse or holds more than one entry; the file name is not `<repo>-<key>.bib`; the key does not follow `Family-Family-Family[-etal]-Year[a-z]` (three names, then `etal`, for four or more authors); the key year differs from the `date` year or lies outside the decade; a required field is missing; a non-ASCII character; an unescaped `%` in a text field; a key or DOI used twice; biber reports a missing mandatory field.
- **WARNING:** an entry type outside the repository's type; a lone suffix (`b` without `a`); other biber datamodel warnings.
- **STYLE:** not in the house layout (see `LibFormat()`); `\a'`-style accents; a page range with one hyphen; a DOI with a prefix; no final newline.

biber's warning that `publisher` is not an `@Article` field is the house convention and is not reported.

Value

Returns a data frame with the columns `file`, `key`, `severity`, `check`, and `message`, invisibly.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
LibCheck(path = getwd())

## End(Not run)
```

`LibFormat` *Put the Entries of a lib-* Library in the House Layout*

Description

Rewrites each entry file in `.setup/latex/bib` of a `lib-*` project in the house layout, without changing what the entry says:

Usage

```
LibFormat(path, write = FALSE)
```

Arguments

`path`	Character string. Project path.
`write`	Logical. If `TRUE`, rewrite the files that change. If `FALSE`, only report them.

Details

- `@Type{Key,`, then one `field = {value},` per line, then `}`, and a final newline;
- fields in the house order: `author`, `editor`, `bookauthor`, `booktitle`, `date`, `journaltitle`, `title`, `subtitle`, the other fields alphabetically, then `abstract`, `publisher`, `keywords`, `library`, and `annotation`;
- field names in lower case and entry types in the house spelling (`Article`, `InBook`, `PhdThesis`, ...);
- `\a'`-style accents as `\'` (`{\a'e}` becomes `{\'e}`);
- page ranges with `--`;
- DOIs without a `https://doi.org/` or `doi:` prefix.

Files that do not parse, or that hold more than one entry, are left as they are (see `LibCheck()`).

Value

Returns the names of the files that change (or changed), invisibly.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
LibFormat(path = getwd())

## End(Not run)
```

LibPaths *Add* .library *to Library Paths*

Description

- creates a folder .library on the project path if it does not exist
- creates a .gitignore file inside .library if it does not exist
- adds .library to .libPaths()
- sets R_LIBS_USER to .library

Usage

```
LibPaths(path)
```

Arguments

path	Character string. Project path.

Value

Returns the path of .library.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
LibPaths(path = getwd())

## End(Not run)
```

License *Generate License File*

Description

Generate License File

Usage

```
License(path, type = "mit")
```

Arguments

path	Character string. Project path.
type	Character string. License type. Available values include "mit" and "gpl3".

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
License(path = getwd())

## End(Not run)
```

Lint *Lint Project*

Description

Runs `lintr` on the R code, then the house checks of the style guides, and prints one table of findings: file, line, rule, message, and level. The R and C++ checks are warnings until a project sets `lint_error <- TRUE` in `.setup/scripts/project.R`: then they stop `make lint`. The LaTeX and Markdown checks are always warnings. Talks (project type `talk`) run the rules kept in talks only.

Usage

```
Lint(path)
```

Arguments

path	Character string. Project path.

Value

Returns the lints object of lintr, invisibly with the house findings as the data frame in its house attribute.

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Lint(path = getwd())

## End(Not run)

## Not run:
Lint(path = getwd())

## End(Not run)
```

Lock *Record What Built the Project*

Description

Writes .setup/scripts/rproject.lock at make auto, before the project is published. The lock records

- the rProject that is running (rproject_sha, rproject_version);
- the image (image, a registry digest, and image_name), read from the running sif's labels;
- the Posit Package Manager snapshot (snapshot, the day before the image's date, the newest snapshot published when the image was built) for packages the image lacks, and the R version (r_version);
- the packages the project adds in project.R (extra_cran, extra_github), with their versions or commits;
- for a proprietary add-on sif (<name>-plus.sif, built from the public image with Bootstrap: localimage): the sif's name, source, checksum (sif_sha256), and date. CI runs the public image.

make project restores the lock, locally and in GitHub Actions; make upgrade moves rProject forward, and the next make auto records the running image.

Usage

```
Lock(path)
```

Arguments

`path`	Character string. Project path.

Details

docker-* projects have no lock: their recurring image builds take the newest upstream base and the newest rProject.

Nothing is written in rProject itself (its CI builds rProject from source), or when the running rProject has no commit (installed from a local source).

Value

Returns the path of the lock file, invisibly, or `NULL` when nothing was written.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Lock(path = getwd())

## End(Not run)
```

`Log` *Create or Update a Change-Set Log*

Description

Creates `.setup/logs/YYYY-MM-DD-<topic>.md`, the log for one change set (one Overleaf push), and adds a pending row to `.setup/logs/index.md`. The log has the commit message skeleton, a "Pushes" table, and the list of files to update in Overleaf.

Usage

```
Log(
  path,
  topic,
  summary = topic,
  private = file.path(path, ".verify"),
  update = FALSE
)
```

Arguments

path	Character string. Project path.
topic	Character string. Short topic in kebab case (e.g., "log-spec"). The change-set log uses the same topic.
summary	Character string. First line of the commit message.
private	Character string. Path to a clone of the private repository at the pushed commit. By default .verify in the project.
update	Logical. If TRUE, refresh only the file lists of the newest existing log for topic.

Details

The file lists come from comparing the project with a clone of the private repository (by default .verify, kept by .setup/scripts/verify.sh), using the same rules as Verify(). Without a clone, the lists hold TODO.

Value

Returns the path of the log invisibly.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Log(path = getwd(), topic = "my-topic")

## End(Not run)
```

Manual *Package Manual*

Description

Package Manual

Usage

```
Manual(path, project)
```

Arguments

path	Character string. Project path.
project	Character string. Project name.

Value

Called for its side effects; the value is not meaningful.

Examples

```
## Not run:
Manual(path = getwd(), project = "myProject")

## End(Not run)
```

PkgDevel *Install Development Packages from CRAN*

Description

Install Development Packages from CRAN

Usage

```
PkgDevel(path)
```

Arguments

path	Character string. Project path.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
PkgDevel(path = getwd())

## End(Not run)
```

PkgDevelGitHub *Install Development Packages from GitHub*

Description

Install Development Packages from GitHub

Usage

```
PkgDevelGitHub(path)
```

Arguments

path	Character string. Project path.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
PkgDevelGitHub(path = getwd())

## End(Not run)
```

PkgProject *Install Project Packages from CRAN*

Description

Install Project Packages from CRAN

Usage

```
PkgProject(path, pkg)
```

Arguments

path	Character string. Project path.
pkg	Character vector. R packages.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
PkgProject(path = getwd(), pkg = "stats")

## End(Not run)
```

`PkgProjectBioconductor`
Install Project Packages from Bioconductor

Description

Install Project Packages from Bioconductor

Usage

```
PkgProjectBioconductor(path, pkg)
```

Arguments

`path`	Character string. Project path.
`pkg`	Character vector. Bioconductor packages.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
PkgProjectBioconductor(path = getwd(), pkg = "stats")

## End(Not run)
```

PkgProjectGitHub *Install Project Packages from GitHub*

Description

A package found in any library is never upgraded. It is installed when it is missing, or when its `ref` is a commit SHA that differs from the installed commit (`RemoteSha`). Dependencies are installed only when missing.

Usage

```
PkgProjectGitHub(path, pkg, ref = "HEAD")
```

Arguments

`path`	Character string. Project path.
`pkg`	Character vector. GitHub packages.
`ref`	Character vector. Git branch, tag, or commit SHA (only a SHA reinstalls).

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
PkgProjectGitHub(path = getwd(), pkg = "stats")

## End(Not run)
```

PkgProjectVersion *Install Project Packages with Specific Version*

Description

Install Project Packages with Specific Version

Usage

```
PkgProjectVersion(path, pkg, ver)
```

Arguments

path	Character string. Project path.
pkg	Character vector. R packages.
ver	Character vector. Version.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
PkgProjectVersion(path = getwd(), pkg = "stats", ver = "1.0.0")

## End(Not run)
```

PkgRepo *Setup RStudio Package Manager Repository Based on the Current Operating System*

Description

The repository is the Posit Package Manager snapshot of the image's date, so that a package the image lacks matches the image's other packages: the environment variable `RPROJECT_SNAPSHOT` (set in GitHub Actions from the lock), the date of the running sif, the lock's `snapshot`, or `latest`.

Usage

```
PkgRepo(path)
```

Arguments

path	Character string. Project path.

Value

Returns the repository url.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
PkgRepo(path = getwd())

## End(Not run)
```

`Profile` *Setup* `.Rprofile`

Description

Setup `.Rprofile`

Usage

```
Profile(path, project)
```

Arguments

`path`	Character string. Project path.
`project`	Character string. Project name.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Profile(path = getwd(), project = "myProject")

## End(Not run)
```

Project *Setup the project.Rproj file*

Description

Setup the project.Rproj file

Usage

```
Project(path)
```

Arguments

path	Character string. Project path.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Project(path = getwd())

## End(Not run)
```

Public *Check the Public Export*

Description

Checks the working tree that `make auto` is about to publish (after `clone.sh` swaps in the public `.git`): no private files (e.g., `.setup/ai/`, `.setup/logs/`, `AGENTS.md`), no file larger than 50 MB, and no tokens (`SecretScan()`) in the files that would be published. `make auto` calls `Public()` before `git add`, so a failed check stops the push.

Usage

```
Public(path)
```

Arguments

path	Character string. Project path.

Value

Returns the files that would be published, invisibly.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Public(path = getwd())

## End(Not run)
```

`Quarto` *Render Quarto Project*

Description

Render Quarto Project

Usage

```
Quarto(path)
```

Arguments

`path`	Character string. Project path.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Quarto(path = getwd())

## End(Not run)
```

ReadMe *Build README.md*

Description

Build README.md

Usage

```
ReadMe(path)
```

Arguments

path	Character string. Project path.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
ReadMe(path = getwd())

## End(Not run)
```

SecretScan *Scan Project for Secrets*

Description

Searches text files in the project for strings that look like tokens (GitHub, Docker Hub, Quarto Pub, Overleaf, Codecov, OpenAI, Anthropic). Matches are reported with the value masked.

Usage

```
SecretScan(path, max_size = 5e+06, error = TRUE)
```

Arguments

path	Character string. Project path.
max_size	Numeric. Files larger than max_size bytes are skipped.
error	Logical. If TRUE, stop with an error when secrets are found.

Value

Returns a data frame of findings with the columns `file`, `line`, `type`, and `match` (masked).

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
SecretScan(path = getwd())

## End(Not run)
```

`Site` *Build Website*

Description

Build Website

Usage

```
Site(path)
```

Arguments

`path`	Character string. Project path.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Site(path = getwd())

## End(Not run)
```

`SizeCheck` *Check a Project Against Overleaf's Limits*

Description

Measures what goes to Overleaf: every file in the project except local-only folders (`.git`, `.verify`, `.library`, `.Rproj.user`, `detritus`) and build output that `make` writes and Overleaf never gets (`docs`, `pkgdown`, `quarto`, `.setup/build`, `.setup/latex/pdf`, `.setup/quarto/bib`), the local-only data tiers (`.setup/data-external`, `.setup/data-cache`), and images (`.sif`).

Usage

```
SizeCheck(
  path,
  editable = 7,
  text_file = 2,
  file = 50,
  total = 100,
  files = 2000L,
  error = FALSE
)
```

Arguments

`path`	Character string. Project path.
`editable`	Numeric. Limit for all editable text, in MB.
`text_file`	Numeric. Limit for one editable text file, in MB.
`file`	Numeric. Limit for one file, in MB.
`total`	Numeric. Recommended limit for the whole project, in MB.
`files`	Integer. Limit for the number of files.
`error`	Logical. If `TRUE`, stop with an error when a check fails.

Details

- **Editable text** (fails): non-binary files in total above `editable` MB. Overleaf's GitHub sync stops above 7 MB.
- **Text file size** (fails): any non-binary file above `text_file` MB. Overleaf caps editable files at 2 MB.
- **File size** (fails): any file above `file` MB. Overleaf caps uploads at 50 MB.
- **Number of files** (fails): more than `files` files. Overleaf allows 2,000.
- **Total size** (warning): above `total` MB. Overleaf recommends under 100 MB with GitHub sync or git.
- **Duplicates** (warning): files of at least 1 MB with the same content.

The report also lists the largest text files and the largest files.

Value

Returns a data frame of checks with the columns `check`, `status`, and `detail`.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
SizeCheck(path = getwd())

## End(Not run)
```

`Spec` *Create a Spec*

Description

Creates `.setup/ai/specs/YYYY-MM-DD-<topic>.md` from a template. Every change set starts with a spec that the user approves.

Usage

```
Spec(path, topic, title = topic)
```

Arguments

`path`	Character string. Project path.
`topic`	Character string. Short topic in kebab case (e.g., `"log-spec"`). The change-set log uses the same topic.
`title`	Character string. Title of the spec.

Value

Returns the path of the spec invisibly.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Spec(path = getwd(), topic = "my-topic")

## End(Not run)
```

`Style` *Style Project*

Description

Style Project

Usage

```
Style(path)
```

Arguments

`path`	Character string. Project path.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Style(path = getwd())

## End(Not run)
```

`Tags` *Library Tags*

Description

Lists the key, `annotation`, and `keywords` of every entry in the project's Quarto bibliography (`.setup/quarto/bib/quarto.bib`, written by `Bib()`), for the "Library Tags" page of the Quarto site.

Usage

```
Tags(path)
```

Arguments

`path`	Character string. Project path.

Value

Returns a data frame with the columns `key`, `annotation`, and `keywords` (`NA` when an entry has none), sorted by key. Without the bibliography, an empty data frame, with a message.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Tags(path = getwd())

## End(Not run)
```

`Template` *Write the Template Files*

Description

Writes the template files of the project's type from rProject (`inst/template/`), so that a template change reaches a project with `make` and no Overleaf refresh.

Usage

```
Template(path, type = NULL, check = FALSE)
```

Arguments

`path`	Character string. Project path.
`type`	Character string. Project type. If `NULL`, it is read from `.setup/scripts/project.R`.
`check`	Logical. If `TRUE`, write nothing and report what would change.

Details

The type comes from a line in `.setup/scripts/project.R`, `type <- "cran"`; one of `"rProject"`, `"cran"`, `"man"`, `"lib"`, `"talk"`, and `"docker"`. Each type uses a few sets of files (`inst/template/manifest.tsv`). Each file has a tier:

- Tier 1, kept in Overleaf: the files Overleaf needs to compile the PDFs (the LaTeX preamble, `.setup/latex/r-scripts/`, `.setup/load/load.R`, style files, the lib documents).
- Tier 2, written by make: everything else (workflows, scripts, ...). These files are not in Overleaf or the private repository, but the `Makefile` (`make` needs it to run at all) and the `.gitignore` files (a fresh clone keeps its export rules), kept in Overleaf.

Every file is written when it is missing or differs (the template wins; change it in rProject). An overwritten tier 2 file is named; the files kept in Overleaf are named for upload. They are regular files in Overleaf, not links (if one still is, delete the link first); only `.setup/r-dependencies/` link to the functions project. `latexmkrc` and `project.Rproj`, also regular files in Overleaf, are written by `LatexCompile()` and `Project()`.

Besides the files of its sets, a project gets:

- The workflows it chooses from the catalog (`inst/template/workflows/`), named in `project.R`: for example `workflows <- c("docker-build-push-multi")`. The daily docker workflow (the one with `{{title}}`) is written as `<workflow>-<workflow_daily>.yml`, titled `workflow_title`. The daily builds run as a chain: an image with `workflow_cron` runs on that schedule; every image also starts when the image before it sends a `base-image-built` event; and an image with `workflow_next` (`"owner/repository"`, or several) sends that event to each next image when its build succeeds (with the `PAT` secret).
- The folder skeleton of its type: an empty `.gitignore` in each standard folder marked `create` in `inst/template/folders.tsv`, if the folder has none.

Files the template no longer has (`inst/template/removed.tsv`) are deleted and named, so that they can be deleted in Overleaf too. A folder on that list is deleted only when nothing but `.gitignore` files is left in it; otherwise it is named and kept. Nothing in a private folder is deleted.

In rProject itself, the tier 1 files are copied into `inst/template/` first, and the files are written from `inst/template/` in the project folder, not from the installed package.

Nothing is written or deleted in GitHub Actions (`GITHUB_ACTIONS` is `"true"`): CI uses the files already in the public repository.

Value

Returns the paths of the files that were written (with `check = TRUE`, that would be written), invisibly, with the deleted files (or those that would be deleted) in the attribute `removed`.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Template(path = getwd())

## End(Not run)
```

TinyTex *Install ~/.TinyTex*

Description

Install ~/.TinyTex

Usage

```
TinyTex(path, bundle = "TinyTeX-2", force = FALSE)
```

Arguments

path	Character string. Project path.
bundle	The bundle name of TinyTeX (which determines the collection of LaTeX packages to install). See https://github.com/rstudio/tinytex-releases#releases for all possible bundles and their meanings.
force	Logical. Force install.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
TinyTex(path = getwd())

## End(Not run)
```

Tree *Project Folders and What They Are For*

Description

Prints the standard folders of the project's type (inst/template/folders.tsv) with their purpose, and flags the folders that differ from the standard: a folder that make creates for the type but that is missing, and a folder that is not standard for the type.

Usage

```
Tree(path)
```

Arguments

path	Character string. Project path.

Details

The type comes from the type line in .setup/scripts/project.R (see Template()); without it, it is guessed as in AI().

Value

Returns a data frame, invisibly, with the columns folder, status ("present", "missing", "absent" for an optional folder, "not standard" for this type, or "unknown"), private, and purpose.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Tree(path = getwd())

## End(Not run)
```

Verify *Verify an Overleaf Push*

Description

Checks a push from Overleaf to the private GitHub repository against the change-set log in .setup/logs.

Usage

```
Verify(
  path,
  private = file.path(path, ".verify"),
  log = NULL,
  record = FALSE,
  error = TRUE
)
```

Arguments

path	Character string. Project path.
private	Character string. Path to a clone of the private repository at the pushed commit. By default .verify in the project.

log	Character string. File name of the change-set log in .setup/logs. If NULL, the log named in the pushed commit message (Log: .setup/logs/<log>), else the newest log with a pending push in .setup/logs/index.md.
record	Logical. If TRUE, write the commit and the result in the log's "Pushes" table and in .setup/logs/index.md, filling the pending row, or adding a row when there is none.
error	Logical. If TRUE, stop with an error when a check fails.

Details

The checks, and when each fails:

- **New commit:** the commit is already recorded in index.md (.verify is stale, or there was no new push). Nothing is recorded.
- **Listed files:** a file listed under "Files to update in Overleaf" is missing from or differs in the private repository. The list is the one of the pushed commit: the section of the log with a code block that starts with the commit's first line (the top-level lists for the log's own commit message; the lists inside a later push's ## section, none for a push of records only). If no block matches, every list in the log is used.
- **Deletions:** a file listed under "Delete in Overleaf" is still in the private repository.
- **Unlisted changes:** (warning only) other files differ between the project and the private repository (.setup/notes/claude.txt, the resume commands, and .setup/todo/, the user's to-do list, are not counted).
- **Commit message:** (warning only) the last commit message does not end with Log: .setup/logs/<log>.
- **Secrets:** SecretScan() finds a token in the private repository.
- **Public export:** a file that make auto would publish is private (e.g., .setup/ai/, .setup/logs/, .setup/repo/, AGENTS.md) or larger than 50 MB.
- **Repo info:** .setup/repo/private-overleaf.md is not an Overleaf project URL (for example, a share link, which grants access). Warning only: a file in .setup/repo/ is missing, private-github.md differs from the .verify remote, or public-github.md differs from the repository in clone.sh.

Generated files (e.g., NAMESPACE, man/), local folders (e.g., .library), and secret files are never compared.

The private repository must be cloned on the host, because SSH keys are not available in the container. .setup/scripts/verify.sh clones or pulls it into .verify (local only; never uploaded) and then calls Verify().

Value

Returns a data frame of checks with the columns check, status, and detail.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
Verify(path = getwd())

## End(Not run)
```

`VignettesPrecompile` *Precompile Vignettes*

Description

Precompile Vignettes

Usage

```
VignettesPrecompile(path)
```

Arguments

`path`	Character string. Project path.

Value

Called for its side effects; the value is not meaningful.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
## Not run:
VignettesPrecompile(path = getwd())

## End(Not run)
```

z *Data from the Standard Normal Distribution*

Description

Data from the Standard Normal Distribution

Usage

```
z
```

Format

A vector of length 100.

Index

∗ **data**
    z, 52

AI, 3
AI(), *49*
AIGlobal, 4
AIGlobal(), *3*

Bib, 5
Bib(), *45*
BibLib, 6
Binary, 7
Build, 7
BuildIgnore, 8

CFF, 9
Clean, 10
CleanPkg, 10
CleanProj, 11
CleanTinyTex, 12
ConfigFiles, 13
Coverage, 13

DataAnalysis, 14
DataCompress, 15
DataManifest, 15
DataPath, 16
DataPath(), *15*
DataProcess, 17
Dependencies, 18
Detritus, 18

Environment, 19
EnvironmentProject, 20
EnvironmentTokens, 20

InstallLocal, 21

LatexCompile, 22
LatexKnit, 23
LatexMake, 23
LatexTikzCompile, 24
LibAdd, 25
LibCheck, 26
LibCheck(), *5*, *25*, *27*
LibFormat, 27
LibFormat(), *26*
LibPaths, 28
License, 29
Lint, 29
Lock, 30
Log, 31

Manual, 32

PkgDevel, 33
PkgDevelGitHub, 34
PkgProject, 34
PkgProjectBioconductor, 35
PkgProjectGitHub, 36
PkgProjectVersion, 36
PkgRepo, 37
Profile, 38
Project, 39
Project(), *4*
Public, 39

Quarto, 40

ReadMe, 41

SecretScan, 41
SecretScan(), *39*, *50*
Site, 42
SizeCheck, 43
SizeCheck(), *15*
Spec, 44
Style, 45

Tags, 45
Template, 46
Template(), *3*, *49*

TinyTex, 48
Tree, 48

utils::read.table(), *16*

Verify, 49
Verify(), *32*
VignettesPrecompile, 51

z, 52